Développement web full-stack

Informatique — Licence 2 (L2)

Cours INF-L2-08

Institut Africain de Management (IAM) — Bamako, Mali
Bibliothèque Universitaire — Bronbah Tech

À propos de ce cours

Construire une application web complète, de bout en bout : architecture client-serveur et protocole HTTP, JavaScript moderne et appels fetch côté client, logique métier et accès aux données côté serveur (PHP/MySQL), authentification et rôles, sécurité de base (XSS, CSRF, injections) et déploiement en production.

Document pédagogique original créé pour les étudiants de l'IAM : leçons détaillées, exemples commentés, travaux dirigés et pratiques, exercices corrigés, QCM d'évaluation et vidéos recommandées. Aucun contenu n'est reproduit d'ouvrages existants.

Sommaire

• Introduction à la matière
• Objectifs d'apprentissage
• Leçon 1 — Architecture client-serveur et protocole HTTP en pratique
• Leçon 2 — Structurer un projet web : dossiers et conventions
• Leçon 3 — Côté client : JavaScript moderne et appels asynchrones avec fetch
• Leçon 4 — Côté serveur : logique métier et accès aux données
• Leçon 5 — Authentification et gestion des rôles utilisateurs
• Leçon 6 — Sécurité web de base : XSS, CSRF et injections
• Leçon 7 — Déploiement et mise en production d'un site
• Travaux dirigés (TD)
• Travaux pratiques (TP)
• Exercices corrigés supplémentaires
• QCM d'évaluation
• Résumé — les notions importantes
• Vidéos associées aux leçons
• Sources et références

Comment utiliser ce cours : lis chaque leçon dans l'ordre, refais les exemples à la main, tente chaque exercice AVANT de lire son corrigé, puis termine par le QCM pour vérifier ce que tu as retenu.

Introduction à la matière

En Licence 1, tu as construit des pages HTML, mis en forme avec CSS, ajouté de l'interactivité avec JavaScript, puis créé des pages dynamiques en PHP reliées à MySQL. Ce cours assemble toutes ces briques en une seule application complète, dite « full-stack » : un client qui tourne dans le navigateur, un serveur qui exécute la logique métier, et une base de données qui conserve les informations. C'est exactement l'architecture de presque tous les services que tu utilises chaque jour, des réseaux sociaux aux applications bancaires, en passant par les plateformes de cours en ligne et les portails d'inscription des universités. Comprendre cette architecture, c'est comprendre le web moderne dans son ensemble.

Notre fil rouge sera BiblioIAM, l'application de gestion des emprunts de la bibliothèque de l'institut : les étudiants consultent le catalogue et réservent des livres, les bibliothécaires enregistrent les emprunts et les retours. Nous la suivrons de la première requête HTTP jusqu'à sa mise en production, en passant par l'organisation du code, l'API, l'authentification et la sécurité. Chaque leçon ajoute une couche réelle à l'application, avec du code que tu peux taper et tester tel quel. Tu verras que ce fil rouge n'a rien d'artificiel : catalogue, comptes, rôles et règles de prêt forment un cas d'école que l'on retrouve, à peine déguisé, dans la plupart des applications de gestion construites pour des entreprises ou des administrations.

Le cours suppose que tu maîtrises les bases de HTML/CSS/JavaScript et de PHP/MySQL vues précédemment. Il ne s'agit plus d'apprendre un langage, mais d'apprendre un métier : découper un problème entre client et serveur, faire dialoguer les deux moitiés

en JSON, protéger l'ensemble, puis le livrer aux utilisateurs. À la fin, tu sauras expliquer, ligne par ligne, ce qui se passe entre le clic d'un étudiant sur « Emprunter » et la ligne qui apparaît dans la base de données. C'est précisément ce savoir-faire — relier toutes les couches sans en négliger aucune — que les recruteurs attendent d'un développeur web, et que le TP final te fera pratiquer de bout en bout.

Objectifs d'apprentissage

À la fin de ce cours, tu seras capable de :

- Décrire l'architecture client-serveur et le déroulement complet d'un échange HTTP (requête, réponse, en-têtes, codes de statut).
- Structurer un projet web full-stack en dossiers clairs (public, api, src, config) avec des conventions de nommage cohérentes.
- Consommer une API depuis JavaScript avec fetch et async/await, en gérant les erreurs réseau et les codes de statut.
- Écrire un point d'entrée d'API en PHP qui applique la logique métier et interroge MySQL via PDO et des requêtes préparées.
- Mettre en place une authentification par session avec mots de passe hachés et une gestion des rôles (étudiant, bibliothécaire).
- Identifier et corriger les failles de base : XSS, CSRF et injection SQL, et appliquer l'échappement des sorties.
- Préparer et réaliser un déploiement en production : environnements, variables de configuration, HTTPS et check-list de mise en ligne.

Leçon 1 — Architecture client-serveur et protocole HTTP en pratique

Une application web repose sur un partage des rôles appelé architecture client-serveur. Le client — le navigateur de l'utilisateur — affiche l'interface, recueille les saisies et envoie des demandes. Le serveur — un programme qui écoute en permanence, comme Apache ou Nginx relayant vers PHP — reçoit ces demandes, applique la logique métier (les règles de gestion : « un étudiant peut emprunter au plus trois livres »), interroge la base de données, puis renvoie une réponse. Ni le navigateur ni la base ne se parlent directement : tout transite par le serveur, qui reste le seul maître des données et des règles.

Le dialogue suit le protocole HTTP, un protocole de texte simple fondé sur un couple requête/réponse. La requête contient une méthode (le verbe : GET pour lire, POST pour créer, PUT ou PATCH pour modifier, DELETE pour supprimer), l'URL de la ressource visée, des en-têtes (Host, Accept, Content-Type, Cookie...) et parfois un corps, par exemple les données d'un formulaire encodées en JSON. La réponse contient un code de statut qui résume l'issue : 200 OK, 201 Created après une création réussie, 400 Bad Request si la demande est mal formée, 401 Unauthorized si l'utilisateur n'est pas connecté, 403 Forbidden s'il n'a pas le droit, 404 Not Found si la ressource n'existe pas, 500 Internal Server Error si le serveur a planté.

HTTP est dit « sans état » (stateless) : chaque requête est indépendante, le serveur ne se souvient pas de la précédente. C'est le navigateur qui renvoie automatiquement, à chaque requête, les cookies que le serveur lui a confiés ; parmi eux, le cookie de session permettra au serveur de reconnaître l'utilisateur connecté (leçon 5). Retiens aussi la différence entre les deux transports : HTTP transmet en clair, lisible par tout intermédiaire, tandis que HTTPS chiffre l'échange grâce à TLS — indispensable dès qu'un mot de passe ou une donnée personnelle circule, donc obligatoire pour toute application réelle.

L'essentiel
Client et serveur se partagent le travail : le navigateur affiche et demande, le serveur décide et accède aux données. HTTP échange des requêtes (méthode + URL + en-têtes + corps éventuel) contre des réponses identifiées par un code de statut. Le protocole est sans état : les cookies assurent la continuité entre les requêtes.

Exemple — Une requête et sa réponse, en texte brut (BiblioIAM)

```
--- REQUETE envoyee par le navigateur ---
GET /api/livres.php?recherche=reseaux HTTP/1.1
Host: biblio.iam-ml.com
Accept: application/json
Cookie: PHPSESSID=8f3k29dksla04mz

--- REPONSE renvoyee par le serveur ---
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Content-Length: 214

[
{"id": 12, "titre": "Reseaux et protocoles",
"auteur": "M. Traore", "disponibles": 3},
{"id": 27, "titre": "TCP/IP illustre",
"auteur": "S. Keita", "disponibles": 0}
]

--- Lecture ---
Methode GET + parametre "recherche" : on LIT des donnees.
Statut 200 : tout s'est bien passe.
Content-Type annonce du JSON : le client sait comment
lire le corps de la reponse.
```

À retenir

- Méthodes : GET lit, POST crée, PUT/PATCH modifie, DELETE supprime — la méthode dit l'intention, l'URL dit la ressource.
- Codes de statut : 2xx succès, 4xx erreur du client (400, 401, 403, 404), 5xx erreur du serveur (500).
- HTTP est sans état : sans cookie de session, le serveur ne reconnaît pas deux requêtes du même utilisateur.

Exercice

Un étudiant connecté clique sur « Emprunter » pour le livre n° 12. Décris la requête HTTP correspondante : méthode, URL plausible, en-têtes indispensables et corps éventuel. Puis donne le code de statut de la réponse dans trois cas : l'emprunt est enregistré ; l'étudiant a déjà trois emprunts en cours (règle métier refusée) ; l'étudiant n'est en fait pas connecté.

✔ Corrigé
Requête : POST /api/emprunts.php, avec l'en-tête Cookie (identifiant de session), Content-Type: application/json, et un corps JSON comme {"livre_id": 12}. On utilise POST car on CRÉE une ressource (un emprunt), pas un GET qui ne doit jamais modifier les données. Cas 1 : 201 Created, avec en corps l'emprunt créé (id, date de retour prévue). Cas 2 : la demande est correcte techniquement mais la règle métier la refuse → 422 Unprocessable Content (ou 409 Conflict ; l'essentiel est un code 4xx cohérent, jamais 200). Cas 3 : 401 Unauthorized — sans session valide, le serveur ne sait pas qui demande, il refuse avant même d'examiner la règle des trois emprunts.

Leçon 2 — Structurer un projet web : dossiers et conventions

Un projet full-stack mélange quatre familles de fichiers : les pages et ressources servies au navigateur (HTML, CSS, JavaScript, images), les points d'entrée de l'API, le code métier partagé (fonctions, classes) et la configuration (paramètres de base de données, secrets). Si tout est mélangé à la racine, le projet devient vite illisible — et dangereux : un fichier de configuration téléchargé par erreur expose le mot de passe de la base à tout le web. La première convention, la plus importante, est donc de séparer ce qui est public de ce qui ne l'est pas : seul le dossier public/ (parfois nommé www ou htdocs) est exposé par le serveur web ; tout le reste vit au-dessus, inaccessible par URL.

À l'intérieur, on organise par responsabilité. Le dossier public/ contient le point d'entrée de l'interface (index.php ou index.html), le dossier assets/ pour CSS, JS et images, et le dossier api/ pour les points d'entrée JSON (livres.php, emprunts.php, connexion.php). Le dossier src/ regroupe le code métier réutilisable : connexion à la base (db.php), fonctions d'authentification, classes Livre et Emprunt. Le dossier config/, hors du public, contient les paramètres ; un fichier .env, jamais versionné, contient les secrets. Les noms de fichiers suivent une convention constante — minuscules, mots séparés par des tirets ou des underscores, jamais d'espaces ni d'accents — car les URL sont sensibles à la casse sur un serveur Linux.

Ces conventions ne sont pas de la bureaucratie : elles rendent le projet prévisible. Un développeur qui rejoint l'équipe sait immédiatement où chercher le code qui gère les emprunts, où modifier le pied de page, où se règle l'accès à la base. Elles préparent aussi le déploiement : l'environnement de production aura exactement la même arborescence, seuls les paramètres du .env changeront. Enfin, le projet est versionné avec Git : le code source est suivi, mais le .env, les journaux et les fichiers téléversés sont exclus par un fichier .gitignore — on ne met jamais un secret dans un dépôt.

L'essentiel
Un seul dossier public exposé par le serveur web ; le code métier dans src/, la configuration hors du public, les secrets dans un .env jamais versionné. Des conventions de nommage constantes rendent le projet prévisible pour toute l'équipe et identique entre développement et production.

Exemple — Arborescence du projet BiblioIAM

```
biblio-iam/
|
+-- public/ <- SEUL dossier expose par le serveur
| +-- index.php page d'accueil / catalogue
| +-- assets/
| | +-- css/style.css
| | +-- js/app.js appels fetch vers l'API
| | +-- img/logo.png
| +-- api/
| +-- livres.php GET : catalogue, recherche
| +-- emprunts.php POST : creer un emprunt
| +-- connexion.php POST : se connecter
|
+-- src/ <- code metier, NON accessible par URL
| +-- db.php connexion PDO a MySQL
| +-- auth.php sessions et roles
| +-- emprunt.php regles metier des emprunts
|
+-- config/
| +-- parametres.php constantes non sensibles
+-- .env secrets : JAMAIS versionne
+-- .gitignore exclut .env, logs/, uploads/
+-- README.md installation et lancement
```

À retenir

- Seul public/ est servi par le serveur web : tout fichier placé ailleurs est protégé des téléchargements directs.
- On organise par responsabilité : api/ pour les points d'entrée JSON, src/ pour le métier, config/ pour les réglages.
- .env contient les secrets, .gitignore l'exclut du dépôt : un mot de passe commité est un mot de passe compromis.

Exercice

Dans un projet désorganisé, on trouve à la racine du site, tous accessibles par URL : motdepasse-base.txt, sauvegarde.sql, index.php, style.css, test-connexion.php (qui affiche les erreurs PHP détaillées) et photos-etudiants/. Liste les problèmes de sécurité et d'organisation, puis propose la réorganisation conforme aux conventions du cours.

✔ Corrigé
Problèmes : 1) motdepasse-base.txt téléchargeable par n'importe qui → identifiants de base compromis ; 2) sauvegarde.sql expose tout le contenu de la base (y compris, probablement, les mots de passe) ; 3) test-connexion.php en production révèle chemins, versions et parfois des fragments de code utiles à un attaquant ; 4) photos-etudiants/ sans contrôle d'accès si les photos sont privées. Réorganisation : ne laisser dans public/ que index.php, les assets et l'API ; déplacer la configuration dans config/ avec les secrets dans un .env hors dépôt ; sortir la sauvegarde du serveur web (stockage séparé, chiffré) ; supprimer tout fichier de test de la production ; faire passer les fichiers privés par un script PHP qui vérifie la session avant de les servir.

Leçon 3 — Côté client : JavaScript moderne et appels asynchrones avec fetch

Dans une application full-stack moderne, la page HTML sert de coquille : elle charge une première fois, puis JavaScript met à jour son contenu sans la recharger entièrement. Pour cela, le client demande les données au serveur sous forme de JSON, grâce à la fonction fetch. fetch envoie une requête HTTP depuis le navigateur et renvoie une Promesse : le résultat n'arrive pas tout de suite, car le réseau prend du temps. On écrit donc du code asynchrone : la page reste vivante pendant l'attente, et le code de traitement s'exécute quand la réponse arrive.

La syntaxe moderne repose sur async et await : une fonction déclarée async peut suspendre son exécution à chaque await jusqu'à la résolution de la promesse, ce qui rend le code aussi lisible qu'un code synchrone. Deux précautions sont indispensables. D'abord, fetch ne rejette sa promesse qu'en cas d'échec réseau ; une réponse 404 ou 500 arrive normalement, il faut donc tester reponse.ok (ou reponse.status) soi-même avant de traiter le corps. Ensuite, tout l'appel doit être entouré d'un try/catch pour afficher un message propre à l'utilisateur au lieu d'une page figée. Le corps JSON se lit avec await reponse.json(), et pour envoyer des données on passe à fetch un second paramètre : la méthode, l'en-tête Content-Type: application/json et le corps produit par JSON.stringify.

Une fois les données reçues, on les affiche en construisant le DOM par programmation : createElement et textContent. Ce choix n'est pas qu'esthétique : textContent insère le texte comme du texte, jamais comme du HTML — un titre de livre contenant « <script> » s'affichera tel quel sans être exécuté. C'est la première ligne de défense contre le XSS (leçon 6). À l'inverse, innerHTML avec des données venues du serveur est une porte ouverte aux scripts malveillants et ne doit servir qu'avec des contenus de confiance, épurés.

L'essentiel
fetch envoie une requête HTTP et renvoie une promesse ; async/await rend ce code asynchrone lisible. Il faut tester reponse.ok (un 404 n'est pas une erreur réseau) et entourer l'appel d'un try/catch. Les données reçues s'affichent avec textContent, jamais avec innerHTML brut.

Exemple — Charger le catalogue et l'afficher (app.js)

```
// Recupere les livres et les affiche dans la page.
async function chargerLivres(recherche) {
const zone = document.querySelector("#liste-livres");
zone.textContent = "Chargement...";
try {
const url = "api/livres.php?recherche="
+ encodeURIComponent(recherche);
const reponse = await fetch(url);
if (!reponse.ok) {
throw new Error("Erreur serveur : " + reponse.status);
}
const livres = await reponse.json();
zone.textContent = "";
if (livres.length === 0) {
zone.textContent = "Aucun livre trouve.";
return;
}
for (const livre of livres) {
const li = document.createElement("li");
// textContent : le titre est affiche comme du TEXTE,
// jamais interprete comme du HTML (anti-XSS).
li.textContent = livre.titre + " - " + livre.auteur
+ " (disponibles : " + livre.disponibles + ")";
zone.appendChild(li);
}
} catch (erreur) {
zone.textContent =
"Impossible de charger le catalogue. Verifiez votre connexion.";
console.error(erreur);
}
}

document.querySelector("#btn-recherche")
.addEventListener("click", () => {
const mot = document.querySelector("#mot").value;
chargerLivres(mot);
});
```

À retenir

- fetch résout sa promesse même pour un 404 ou un 500 : c'est à toi de vérifier reponse.ok avant de lire le corps.
- Envoyer du JSON : méthode POST, en-tête Content-Type: application/json, corps = JSON.stringify(objet).
- Afficher des données externes avec textContent : c'est simple et c'est la protection de base contre le XSS.

Exercice

Écris la fonction JavaScript emprunterLivre(id) qui envoie à l'API la demande d'emprunt du livre d'identifiant id (POST sur api/emprunts.php, corps JSON {"livre_id": id}), puis affiche « Emprunt confirmé » si le serveur répond 201, « Vous avez déjà trois emprunts » s'il répond 422, et un message générique dans les autres cas, y compris en cas de

panne réseau.

✔ Corrigé
async function emprunterLivre(id) { try { const reponse = await fetch("api/emprunts.php", { method: "POST", headers: { "Content-Type": "application/json" }, body: JSON.stringify({ livre_id: id }) }); if (reponse.status === 201) { afficherMessage("Emprunt confirmé"); } else if (reponse.status === 422) { afficherMessage("Vous avez déjà trois emprunts en cours."); } else if (reponse.status === 401) { afficherMessage("Connectez-vous pour emprunter."); } else { afficherMessage("Le serveur a rencontré un problème, réessayez."); } } catch (e) { afficherMessage("Panne réseau : vérifiez votre connexion."); } }. Points clés : on branche sur le CODE DE STATUT, pas sur le texte de la réponse ; le try/catch couvre uniquement les échecs réseau et les erreurs de parsing ; le serveur reste seul juge de la règle métier — le client ne fait qu'afficher son verdict.

Leçon 4 — Côté serveur : logique métier et accès aux données

Le point d'entrée de l'API — par exemple api/emprunts.php — est le gardien de l'application. Son travail se déroule toujours dans le même ordre : identifier qui appelle (session valide ?), valider ce qui est demandé (les paramètres sont-ils présents, du bon type, dans les bornes ?), appliquer la logique métier (les règles de gestion, ici dans src/emprunt.php), accéder aux données, puis formuler une réponse JSON accompagnée du bon code de statut. Cet ordre n'est pas négociable : on ne touche jamais à la base avant d'avoir identifié l'appelant et validé ses paramètres.

L'accès aux données se fait avec PDO, l'interface d'accès aux bases de PHP, et exclusivement par requêtes préparées : la requête est écrite avec des marqueurs (?) ou des paramètres nommés (:id), puis les valeurs sont transmises séparément. Le moteur de base distingue alors le code SQL des données : une valeur comme « 12; DROP TABLE livres » est traitée comme une simple chaîne, jamais comme un ordre. C'est la protection définitive contre l'injection SQL (leçon 6). La connexion PDO elle-même est créée une seule fois, dans src/db.php, avec le mode d'erreur « exceptions » pour que toute erreur SQL interrompe le traitement au lieu de passer inaperçue.

Le serveur répond toujours en JSON dans une API, jamais en HTML : il pose l'en-tête Content-Type: application/json; charset=utf-8, le code de statut adapté via http_response_code(), et il sérialise les données avec json_encode. En cas d'erreur interne, la réponse doit rester générique (« Une erreur est survenue ») avec un statut 500 : le détail technique (message SQL, chemin de fichier) va dans le journal du serveur, pas chez le client, où il servirait de carte à un attaquant. Un point d'entrée bien écrit est donc ennuyeux et prévisible — et c'est un compliment.

L'essentiel
Un point d'entrée d'API suit toujours le même ordre : authentifier, valider les entrées, appliquer la logique métier, accéder aux données via PDO en requêtes préparées, puis répondre en JSON avec le bon code de statut. Les détails d'erreur restent dans les journaux du serveur.

Exemple — Point d'entrée de recherche du catalogue (api/livres.php)

```
<?php
// api/livres.php -- GET : rechercher des livres du catalogue.
declare(strict_types=1);
require __DIR__ . "/../src/db.php";

header("Content-Type: application/json; charset=utf-8");

// 1. Valider l'entree : terme optionnel, longueur bornee.
$recherche = trim($_GET["recherche"] ?? "");
if (mb_strlen($recherche) > 100) {
http_response_code(400);
echo json_encode(["erreur" => "Recherche trop longue."]);
exit;
}

try {
// 2. Requete preparee : la valeur n'est JAMAIS colle
// dans le SQL, elle voyage separement.
$sql = "SELECT id, titre, auteur, disponibles
FROM livres
WHERE titre LIKE :mot OR auteur LIKE :mot
ORDER BY titre";
$stmt = $pdo->prepare($sql);
$stmt->execute(["mot" => "%" . $recherche . "%"]);
$livres = $stmt->fetchAll(PDO::FETCH_ASSOC);

// 3. Repondre : statut 200 implicite + JSON.
echo json_encode($livres);
} catch (PDOException $e) {
// Le detail va dans le journal, PAS chez le client.
error_log("livres.php : " . $e->getMessage());
http_response_code(500);
echo json_encode(["erreur" => "Une erreur est survenue."]);
}
```

À retenir

• Ordre immuable d'un point d'entrée : authentifier → valider → métier → données → réponse JSON.

• PDO + requêtes préparées partout, sans exception : les valeurs ne sont jamais concaténées dans le SQL.

• Erreur interne = statut 500 et message générique pour le client ; le détail technique part dans error_log.

Exercice

Écris le squelette du point d'entrée api/retours.php (POST) qui enregistre le retour d'un emprunt : il reçoit {"emprunt_id": …}, vérifie que l'appelant est un bibliothécaire, que l'emprunt existe et n'est pas déjà clos, puis met à jour la base. Indique, pour chaque refus possible, le code de statut que tu renvoies et pourquoi.

✔ Corrigé
Squelette : session_start et contrôle du rôle (leçon 5) → 401 si non connecté, 403 si connecté mais simple étudiant. Lecture et validation du JSON : emprunt_id absent ou non entier → 400. Recherche de l'emprunt par requête préparée : introuvable → 404. Emprunt déjà clos (date_retour_reelle renseignée) → 409 Conflict, car la demande contredit l'état actuel de la ressource. Mise à jour par requête préparée (UPDATE emprunts SET date_retour_reelle = CURRENT_DATE WHERE id = :id), recalcul éventuel de la pénalité dans src/emprunt.php, puis réponse 200 avec l'emprunt mis à jour en JSON. Toute PDOException → journal + 500 générique. L'exercice illustre la grille de lecture des statuts : 400 = demande mal formée, 401/403 = problème d'identité ou de droit, 404 = ressource absente, 409 = conflit d'état, 500 = faute du serveur.

Leçon 5 — Authentification et gestion des rôles utilisateurs

L'authentification répond à la question « qui es-tu ? », l'autorisation à la question « as-tu le droit de faire cela ? ». Pour BiblioIAM, l'étudiant se connecte avec son numéro d'étudiant et un mot de passe. Le serveur ne stocke jamais ce mot de passe en clair : il stocke une empreinte calculée par password_hash() (algorithme bcrypt ou Argon2id), une fonction à sens unique et lente par conception. À la connexion, password_verify() compare le mot de passe saisi à l'empreinte stockée, sans jamais pouvoir la « déchiffrer ». Une base volée ne livre donc pas les mots de passe — à condition, évidemment, de ne pas utiliser les vieux hachages rapides comme MD5 ou SHA1, cassés en quelques secondes.

Une fois l'identité vérifiée, le serveur crée une session : il génère un identifiant aléatoire imprévisible, le confie au navigateur dans un cookie marqué HttpOnly (inaccessible à JavaScript, donc protégé du vol par XSS) et conserve côté serveur les informations associées — ici l'identifiant de l'utilisateur et son rôle. Deux gestes sont critiques : appeler session_regenerate_id(true) juste après la connexion, pour invalider tout identifiant fixé avant l'authentification (protection contre la fixation de session), et prévoir une déconnexion qui détruit la session côté serveur, pas seulement le cookie. À chaque requête protégée, le point d'entrée vérifie la présence de la session avant tout traitement — c'est la première étape de l'ordre vu à la leçon 4.

Les rôles organisent ensuite les autorisations. BiblioIAM en compte deux : l'étudiant (consulter le catalogue, emprunter, voir ses emprunts) et le bibliothécaire (enregistrer les retours, gérer le catalogue, voir tous les emprunts). Le rôle est lu dans la session — une donnée côté serveur — et jamais dans un paramètre envoyé par le client : un champ caché « role=admin » dans un formulaire se modifie en deux secondes dans le navigateur. Le contrôle doit être fait par le serveur, sur chaque point d'entrée sensible,

via une petite fonction de garde qui refuse avec un 403. Cacher un bouton dans l'interface n'est qu'un confort d'affichage, jamais une protection.

L'essentiel
Les mots de passe se stockent hachés (password_hash/password_verify), jamais en clair. La session vit côté serveur, son identifiant voyage dans un cookie HttpOnly, régénéré à la connexion. Les rôles se contrôlent côté serveur sur chaque point d'entrée : l'interface peut cacher, seul le serveur peut interdire.

Exemple — Connexion et garde de rôle (api/connexion.php et src/auth.php)

```
<?php
// --- api/connexion.php : POST {"matricule", "mot_de_passe"}
declare(strict_types=1);
require __DIR__ . "/../src/db.php";
header("Content-Type: application/json; charset=utf-8");

$donnees = json_decode(file_get_contents("php://input"), true);
$matricule = trim($donnees["matricule"] ?? "");
$motDePasse = $donnees["mot_de_passe"] ?? "";

$stmt = $pdo->prepare(
"SELECT id, mot_de_passe_hash, role FROM utilisateurs
WHERE matricule = :m");
$stmt->execute(["m" => $matricule]);
$user = $stmt->fetch(PDO::FETCH_ASSOC);

// Meme message si le compte n'existe pas ou si le mot
// de passe est faux : on ne dit pas lequel des deux.
if (!$user || !password_verify($motDePasse, $user["mot_de_passe_hash"])) {
http_response_code(401);
echo json_encode(["erreur" => "Identifiants incorrects."]);
exit;
}

session_start();
session_regenerate_id(true); // anti fixation de session
$_SESSION["user_id"] = (int) $user["id"];
$_SESSION["role"] = $user["role"]; // "etudiant" ou "bibliothecaire"
echo json_encode(["ok" => true, "role" => $user["role"]]);

// --- src/auth.php : la garde, a appeler en tete des
// --- points d'entree proteges.
function exigerRole(string $roleRequis): void {
session_start();
if (!isset($_SESSION["user_id"])) {
http_response_code(401);
echo json_encode(["erreur" => "Non connecte."]);
exit;
}
if ($_SESSION["role"] !== $roleRequis) {
http_response_code(403);
echo json_encode(["erreur" => "Acces reserve : " . $roleRequis]);
exit;
}
}
```

À retenir

• password_hash() pour stocker, password_verify() pour contrôler ; MD5 et SHA1 sont interdits pour les mots de passe.

• À la connexion : session_regenerate_id(true) ; à la déconnexion : destruction de la session côté serveur.

• Le rôle se lit dans la session serveur et se contrôle par point d'entrée — jamais depuis une donnée envoyée par le client.

Exercice

Un collègue propose trois « simplifications » pour BiblioIAM : 1) stocker les mots de passe en SHA1 « parce que c'est déjà un hachage » ; 2) transmettre le rôle dans un champ caché du formulaire de retour, pour éviter de relire la session ; 3) afficher « Ce mot de passe est faux » quand le matricule existe mais pas le mot de passe. Explique le danger précis de chacune et la pratique correcte.

✔ Corrigé
1) SHA1 est un hachage rapide conçu pour l'intégrité des fichiers : un attaquant peut tester des milliards de combinaisons par seconde et retrouver les mots de passe courants d'une base volée. password_hash utilise un algorithme lent avec sel intégré (bcrypt/Argon2id), fait exactement pour rendre cette recherche impraticable. 2) Un champ caché est une donnée du client : n'importe qui peut la modifier avec les outils du navigateur et se déclarer bibliothécaire. Le rôle doit venir de la session, écrite par le serveur seul au moment de la connexion. 3) Distinguer « compte inexistant » et « mot de passe faux » offre un oracle d'énumération : l'attaquant teste des matricules et découvre lesquels correspondent à de vrais comptes, première étape d'une attaque ciblée. Le message doit rester unique et vague : « Identifiants incorrects ».

Leçon 6 — Sécurité web de base : XSS, CSRF et injections

Trois familles de failles concentrent l'essentiel des attaques contre les petites applications web. Le XSS (Cross-Site Scripting) consiste à injecter un script dans une page vue par d'autres : si le titre d'un livre saisi par un utilisateur est réaffiché sans échappement, un titre comme « <script>…</script> » s'exécutera dans le navigateur des visiteurs, par exemple pour voler leur cookie de session. La parade tient en une règle : toute donnée affichée est échappée selon son contexte — htmlspecialchars() côté PHP, textContent côté JavaScript — et le cookie de session est marqué HttpOnly pour limiter les dégâts. L'injection SQL, vue à la leçon 4, colle des données dans une requête ; les requêtes préparées la rendent impossible en séparant code et données.

Le CSRF (Cross-Site Request Forgery) exploite la confiance du serveur dans le navigateur : l'utilisateur, connecté à BiblioIAM, visite un site piégé qui déclenche en arrière-plan une requête vers BiblioIAM — par exemple un formulaire caché envoyé automatiquement. Le navigateur joint le cookie de session, et le serveur exécute l'action comme si l'utilisateur l'avait demandée. La parade standard est le jeton CSRF : le serveur glisse dans chaque formulaire un secret aléatoire lié à la session, et refuse toute action POST dont le jeton est absent ou différent (comparé avec hash_equals). Un site extérieur ne peut pas connaître ce jeton, donc ses requêtes forgées échouent. L'attribut de cookie SameSite=Lax apporte une défense complémentaire en limitant l'envoi du cookie depuis d'autres sites.

Ces trois défenses partagent une philosophie : ne jamais faire confiance aux données qui franchissent la frontière du navigateur — ni à celles qu'il envoie (validation et requêtes préparées côté serveur), ni à celles qu'on lui renvoie (échappement des sorties), ni même à ses requêtes apparemment légitimes (jeton CSRF). Et la validation

côté client, utile pour le confort, ne protège rien : l'attaquant n'utilise pas ton formulaire, il fabrique ses requêtes directement. Toute règle de sécurité doit donc vivre côté serveur ; le client ne fait que de la présentation. C'est cette discipline, plus qu'un outil magique, qui sépare une application « qui marche » d'une application déployable.

L'essentiel
XSS : échapper toute sortie (htmlspecialchars, textContent) et cookie HttpOnly. Injection SQL : requêtes préparées, toujours. CSRF : jeton secret par session, vérifié sur chaque POST, comparé avec hash_equals. Règle d'or : aucune confiance dans ce qui vient du navigateur ; la sécurité se contrôle côté serveur.

Exemple — Vulnérable puis corrigé : les trois réflexes

```
<?php
// ============ 1. XSS ============
// VULNERABLE : le titre saisi par un utilisateur est
// affiche tel quel -> un <script> s'executera chez
// les autres visiteurs.
echo "<li>" . $livre["titre"] . "</li>";
// CORRIGE : echapper a l'affichage.
echo "<li>" . htmlspecialchars($livre["titre"],
ENT_QUOTES, "UTF-8") . "</li>";

// ============ 2. INJECTION SQL ============
// VULNERABLE : la saisie est collee dans la requete.
// id = "1 OR 1=1" afficherait tous les emprunts.
$sql = "SELECT * FROM emprunts WHERE id = " . $_GET["id"];
// CORRIGE : requete preparee, la valeur voyage a part.
$stmt = $pdo->prepare(
"SELECT * FROM emprunts WHERE id = :id");
$stmt->execute(["id" => (int) $_GET["id"]]);

// ============ 3. CSRF ============
// A la connexion (ou a l'affichage du formulaire) :
// $_SESSION["csrf"] = bin2hex(random_bytes(32));
// Dans le formulaire HTML :
// <input type="hidden" name="csrf"
// value="<?= $_SESSION['csrf'] ?>">
// Au traitement de CHAQUE action POST :
function verifierJetonCsrf(): void {
$recu = $_POST["csrf"] ?? "";
if (!hash_equals($_SESSION["csrf"] ?? "", $recu)) {
http_response_code(403);
exit("Jeton CSRF invalide.");
}
}
```

À retenir

• XSS = une donnée devient du code dans le navigateur d'autrui ; on l'empêche en échappant chaque affichage, sans exception.

• CSRF = le navigateur connecté est utilisé comme un pantin ; le jeton secret par session rend la requête forgée irrecevable.

• Validation côté client = confort ; validation et contrôles côté serveur = sécurité. Les deux, mais jamais l'inverse.

Exercice

Voici trois extraits livrés par un camarade pour BiblioIAM. Pour chacun, nomme la faille, explique le scénario d'attaque en une phrase, puis donne le correctif. A) $pdo->query("DELETE FROM livres WHERE id = " . $_POST["id"]). B) Un formulaire de suppression de livre sans aucun champ caché, protégé uniquement parce que « le bouton n'apparaît que pour les bibliothécaires ». C) Le résumé d'un livre, saisi par les bibliothécaires, affiché avec innerHTML dans la page du catalogue.

✔ Corrigé
A) Injection SQL (avec, en prime, une action destructive sans contrôle de rôle) : un id forgé peut transformer le DELETE en destruction massive. Correctif : requête préparée avec paramètre lié, conversion en entier, et garde exigerRole('bibliothecaire') en tête du point d'entrée. B) CSRF : l'absence du bouton n'empêche pas un site piégé de faire envoyer le POST par le navigateur d'un bibliothécaire connecté — il n'y a rien à « cliquer », la requête part toute seule avec le cookie. Correctif : jeton CSRF dans le formulaire, vérifié côté serveur avec hash_equals ; le masquage du bouton reste un confort d'affichage. C) XSS stocké : un résumé contenant un script s'exécutera chez chaque visiteur du catalogue, avec accès potentiel à leurs actions. Correctif : afficher le résumé avec textContent (ou htmlspecialchars côté serveur si le HTML est généré en PHP) ; ne réserver innerHTML qu'à des contenus de confiance préalablement épurés par une bibliothèque de nettoyage dédiée.

Leçon 7 — Déploiement et mise en production d'un site

Développer « chez soi » et servir de vrais utilisateurs sont deux métiers. Un projet sérieux vit dans au moins deux environnements : le développement (ta machine, données de test, erreurs affichées pour déboguer) et la production (le serveur public, vraies données, erreurs masquées et journalisées). On ajoute souvent un environnement de préproduction, copie de la production pour répéter la mise en ligne. La règle qui rend ces environnements gouvernables : le code est identique partout, seuls les paramètres changent — et ils vivent dans le fichier .env de chaque machine (adresse de la base, identifiants, mode debug activé ou non). Ainsi, corriger un bug en développement puis déployer le même code en production ne demande aucune retouche.

Le déploiement lui-même suit un rituel. On fige une version du code (un commit Git étiqueté), on sauvegarde la base de production avant toute modification de sa structure, on transfère le code (git pull sur le serveur, ou envoi des fichiers), on applique les migrations de base de données, puis on vérifie une liste de points critiques : page d'accueil, connexion, une action métier complète. L'hébergement peut être mutualisé (simple, limité, fréquent pour PHP/MySQL), un VPS que l'on administre

soi-même, ou une plateforme qui automatise le déploiement à chaque push. Pour BiblioIAM, un hébergement PHP/MySQL classique suffit largement — l'important n'est pas l'outil mais la répétabilité : un déploiement doit pouvoir être refait à l'identique, et annulé si quelque chose casse.

La mise en production se termine par des réglages qui n'existent pas en développement. Le HTTPS est activé (certificat gratuit Let's Encrypt) et le HTTP redirigé vers lui, car les mots de passe et cookies ne doivent jamais voyager en clair. Les erreurs PHP sont masquées (display_errors désactivé) mais journalisées, le mode debug de l'application est éteint, les comptes par défaut sont supprimés et les mots de passe de la base sont spécifiques à la production — jamais ceux du poste de développement. Enfin, on met en place le minimum vital d'exploitation : sauvegardes automatiques de la base, et un œil régulier sur les journaux. Un site en production n'est pas un site « fini » : c'est un site dont on a organisé la surveillance et les retours en arrière.

L'essentiel
Même code partout, paramètres par environnement dans le .env. Déploiement = rituel répétable : version figée, sauvegarde de la base, transfert, migrations, vérifications, retour arrière possible. Production = HTTPS obligatoire, erreurs masquées et journalisées, secrets propres à la production, sauvegardes automatiques.

Exemple — Configuration par environnement et check-list de mise en ligne

```
# --- Fichier .env sur le SERVEUR DE PRODUCTION
# (jamais versionne, jamais celui du poste de dev)
APP_ENV=production
APP_DEBUG=false
DB_HOST=localhost
DB_NAME=biblio_iam_prod
DB_USER=biblio_app
DB_PASS=mot-de-passe-long-et-unique-a-generer

# --- src/config.php (code identique partout)
// Lit le .env et expose les parametres a l'application.
// En developpement, APP_DEBUG vaut true et les erreurs
// s'affichent ; en production, elles partent dans les
// journaux et l'utilisateur ne voit qu'un message sobre.

# --- CHECK-LIST DE MISE EN PRODUCTION ---
# [ ] Code fige : commit Git etiquete (ex. v1.0.0)
# [ ] Base de production sauvegardee AVANT migration
# [ ] Migration executee et controlee sur copie
# [ ] .env de production en place, secrets renouveles
# [ ] APP_DEBUG=false, display_errors desactive
# [ ] HTTPS actif, redirection HTTP -> HTTPS verifiee
# [ ] Cookie de session : HttpOnly + Secure + SameSite
# [ ] Comptes de test et acces par defaut supprimes
# [ ] Parcours complet teste : accueil, connexion,
# recherche, emprunt, retour, deconnexion
# [ ] Sauvegarde automatique de la base planifiee
# [ ] Procedure de retour arriere notee et testee
```

À retenir

• Un seul code pour tous les environnements ; ce qui change (base, debug, secrets) vit dans le .env de chaque machine.

• Jamais de déploiement sans sauvegarde préalable de la base ni plan de retour arrière.

• Production : HTTPS forcé, erreurs cachées mais journalisées, debug éteint, mots de passe propres à la production.

Exercice

Le jour du lancement de BiblioIAM, l'équipe constate trois problèmes : les étudiants voient défiler des messages d'erreur SQL détaillés ; la connexion se fait en HTTP simple et le navigateur affiche « Non sécurisé » ; et personne ne sait quelle version du code tourne, car les fichiers ont été copiés un à un par FTP « au fil des corrections ». Pour chaque problème, explique le risque concret et le réglage ou la pratique du cours qui l'aurait évité.

✔ Corrigé
1) Erreurs SQL affichées : display_errors et/ou APP_DEBUG sont restés actifs en production. Risque : ces messages révèlent la structure des tables et des requêtes, un mode d'emploi pour l'injection. Réglage : APP_DEBUG=false dans le .env de production, erreurs journalisées via error_log, message générique pour l'utilisateur. 2) HTTP sans chiffrement : mots de passe et cookies de session circulent en clair, interceptables sur un réseau partagé (Wi-Fi public). Réglage : certificat HTTPS (Let's Encrypt), redirection forcée HTTP → HTTPS, cookie de session avec attributs Secure et HttpOnly. 3) Copie FTP fichier par fichier : aucune version identifiable, aucun retour arrière propre, risque de mélanger ancien et nouveau code. Pratique : déployer depuis Git (clone/pull d'un commit étiqueté), ce qui rend chaque version identifiable et le retour à la version précédente immédiat. Les trois problèmes ont la même racine : la production a été traitée comme un gros poste de développement ; le cours impose l'inverse — deux environnements, un rituel, une check-list.

Travaux dirigés (TD)

TD 1 — Concevoir l'API complète de BiblioIAM

Avant d'écrire le code, l'équipe veut le contrat complet de l'API de BiblioIAM. Établis le tableau des points d'entrée nécessaires pour ces fonctionnalités : consulter le catalogue avec recherche ; voir le détail d'un livre ; se connecter et se déconnecter ; emprunter un livre ; voir « mes emprunts » ; enregistrer un retour ; ajouter un livre au catalogue ; supprimer un livre. Pour chaque point d'entrée, donne : la méthode HTTP, l'URL, le rôle autorisé (public, étudiant, bibliothécaire), les données envoyées et les deux codes de statut principaux (succès, refus le plus probable). Justifie le choix de la méthode pour l'emprunt et pour la suppression.

✔ Corrigé du TD 1
Tableau attendu (les URL exactes peuvent varier, la logique doit être celle-ci) : 1) GET /api/livres.php?recherche= — public — aucune donnée — 200 / 400 si paramètre anormalement long. 2) GET /api/livres.php?id= — public — 200 / 404 livre inexistant. 3) POST /api/connexion.php — public — JSON {matricule, mot_de_passe} — 200 / 401 identifiants incorrects (message unique). 4) POST /api/deconnexion.php — connecté — 200 / 401 si aucune session. 5) POST /api/emprunts.php — étudiant — JSON {livre_id} — 201 / 422 règle des trois emprunts (ou 409), 401 si non connecté. 6) GET /api/emprunts.php?moi=1 — étudiant — 200 / 401 ; le serveur filtre sur l'utilisateur de la session, jamais sur un id fourni par le client. 7) POST /api/retours.php — bibliothécaire — JSON {emprunt_id} — 200 / 403 si étudiant, 404 emprunt inconnu, 409 déjà clos. 8) POST /api/livres.php — bibliothécaire — JSON du livre — 201 / 400 données invalides, 403 si étudiant. 9) DELETE /api/livres.php?id= — bibliothécaire — 200 (ou 204) / 403, 404. Justifications : l'emprunt CRÉE une ressource → POST (GET ne doit jamais modifier l'état) ; la suppression retire une ressource identifiée → DELETE ; dans les deux cas, le rôle est contrôlé côté serveur avant tout accès à la base.

TD 2 — Audit de sécurité d'un point d'entrée livré trop vite

Voici le point d'entrée de suppression d'un livre, écrit dans l'urgence (il est volontairement truffé de fautes). Trouve au moins cinq problèmes, classe-les par gravité, et réécris le point d'entrée corrigé. Code livré : <?php $id = $_GET["id"]; $pdo->query("DELETE FROM livres WHERE id = " . $id); echo "Livre supprime"; ?> — accessible par un simple lien Supprimer dans la page du catalogue, page que seuls les bibliothécaires voient dans leur menu.

✔ Corrigé du TD 2
Problèmes, du plus grave au moins grave : 1) Injection SQL : l'id est concaténé dans la requête ; id = « 0 OR 1=1 » viderait la table des livres. 2) Aucune authentification ni contrôle de rôle : « caché dans le menu » ne protège pas l'URL, appelable directement par n'importe qui, même non connecté. 3) Action destructive en GET : outre le principe (GET ne modifie rien), cela rend l'attaque CSRF triviale — une simple image <img src=".../supprimer.php?id=12"> sur un site piégé, visitée par un bibliothécaire connecté, déclenche la suppression avec son cookie. 4) Pas de jeton CSRF (conséquence du point 3). 5) Aucune validation (id non entier, livre inexistant non géré → pas de 404), aucune trace dans les journaux, réponse en texte brut sans statut maîtrisé. Version corrigée : méthode POST (ou DELETE) exigée ; session_start + exigerRole("bibliothecaire") ; jeton CSRF vérifié par hash_equals ; id converti en entier et testé ; suppression par requête préparée ; si aucune ligne affectée → 404 ; sinon 200 JSON {"ok": true} ; journalisation de l'action (qui a supprimé quoi, quand). L'audit illustre la défense en profondeur : chaque couche (méthode, session, rôle, jeton, préparation, validation) arrête une classe d'attaques différente.

Travaux pratiques (TP)

TP 1 — Construire et déployer BiblioIAM en binôme (3 séances)

En binôme, réalisez BiblioIAM complète puis mettez-la en ligne sur l'hébergement de TP fourni par l'enseignant. Séance 1 — Structure et lecture : créez l'arborescence du cours (public/, src/, config/, .env local, .gitignore), la base MySQL (livres, utilisateurs, emprunts) avec un jeu de données de test, src/db.php en PDO, et le point d'entrée GET du catalogue ; côté client, la page de recherche en fetch avec affichage en textContent. Séance 2 — Écriture et sécurité : connexion avec password_hash/password_verify et session régénérée, garde de rôle, POST d'emprunt avec la règle des trois livres, retour réservé aux bibliothécaires ; appliquez partout requêtes préparées, échappement des affichages et jeton CSRF sur les formulaires d'action. Séance 3 — Production : déployez depuis Git sur l'hébergement (commit étiqueté v1.0), créez le .env de production, activez HTTPS, passez APP_DEBUG à false, exécutez la check-list du cours en testant le parcours complet avec deux comptes (un étudiant, un bibliothécaire). Livrables : le dépôt Git, l'URL publique, et un compte rendu d'une page listant les contrôles de sécurité appliqués, leçon par leçon.

✔ Attendus du TP 1
Attendus et points de vigilance pour l'évaluation : l'arborescence respecte la séparation public/non-public et aucun secret n'apparaît dans l'historique Git (vérifier avec git log) ; le catalogue répond en JSON avec Content-Type correct et la recherche passe par une requête préparée (tester une recherche contenant « ' OR '1'='1 » : elle ne doit retourner que les livres correspondants, sans erreur) ; la règle des trois emprunts est refusée côté serveur avec un 422 même en appelant l'API directement, hors interface ; un étudiant qui appelle le point d'entrée de retour reçoit 403 ; les mots de passe en base commencent par $2y$ ou $argon2 (preuve du hachage) ; le cookie de session est HttpOnly et l'identifiant change après connexion ; un titre de livre contenant <script>alert(1)</script> s'affiche comme du texte ; en production, HTTPS est actif, une erreur provoquée volontairement n'affiche aucun détail SQL, et la check-list signée accompagne le compte rendu. Barème indicatif : architecture et API 30 %, authentification et rôles 25 %, sécurité 25 %, déploiement et compte rendu 20 %.

Exercices corrigés supplémentaires

Exercice 1

Lis cette réponse reçue après l'envoi du formulaire d'emprunt : « HTTP/1.1 201 Created — Content-Type: application/json — Set-Cookie: PHPSESSID=nouvel-id; HttpOnly; Secure; SameSite=Lax — Corps : {"emprunt_id": 57, "date_retour": "2026-11-02"} ». Explique chaque élément : pourquoi 201 et pas 200, le rôle du Set-Cookie et de ses trois attributs, et ce que le client doit faire du corps JSON.

✔ Corrigé
201 Created signale qu'une ressource nouvelle (l'emprunt n° 57) a été créée par un POST : 200 aurait convenu pour une simple lecture ou mise à jour, 201 est le statut précis des créations réussies. Set-Cookie installe (ou renouvelle, après régénération à la connexion) l'identifiant de session que le navigateur renverra automatiquement aux requêtes suivantes ; HttpOnly interdit sa lecture par JavaScript (barrière anti-vol par XSS), Secure impose son envoi uniquement en HTTPS, SameSite=Lax limite son envoi lors de requêtes venues d'autres sites (barrière anti-CSRF). Le corps JSON donne au client de quoi mettre à jour l'affichage sans recharger : ici, afficher la confirmation avec la date de retour du 2 novembre 2026, et idéalement décrémenter le nombre d'exemplaires disponibles affiché pour ce livre.

Exercice 2

Ce code client « fonctionne » lors des démonstrations mais casse en conditions réelles. Trouve ses trois défauts et réécris-le : async function reserver(id) { const r = await fetch("api/emprunts.php?id=" + id); const d = await r.json(); zone.innerHTML = "Emprunt : " + d.message; }

✔ Corrigé
Défaut 1 : l'emprunt est demandé en GET (paramètre dans l'URL, aucune méthode POST) — un GET ne doit jamais modifier l'état ; il peut être mis en cache, préchargé par le navigateur ou rejoué, créant des emprunts fantômes ; et il expose l'action au CSRF. Défaut 2 : aucun test de r.ok ni de branchement sur le statut — un 401, un 422 ou un 500 seront traités comme un succès et d.message sera probablement undefined. Défaut 3 : innerHTML avec une donnée du serveur → porte ouverte au XSS si le message contient un jour une donnée saisie. De plus, pas de try/catch : une panne réseau rejette la promesse sans retour pour l'utilisateur. Version corrigée : POST avec corps JSON et en-tête Content-Type, test des statuts (201 → confirmation ; 401 → inviter à se connecter ; 422 → limite atteinte ; sinon message d'erreur générique), affichage par textContent, le tout dans un try/catch qui signale la panne réseau.

Exercice 3

Pour chaque situation de BiblioIAM, choisis le code de statut le plus juste et justifie en une phrase : a) un visiteur non connecté demande la liste de ses emprunts ; b) un étudiant connecté demande la page d'administration du catalogue ; c) le formulaire d'ajout de livre arrive sans le champ « titre » ; d) on demande le détail du livre n° 9999, qui n'existe pas ; e) le serveur de base de données est en panne pendant une recherche.

✔ Corrigé
a) 401 Unauthorized : le serveur ne sait pas qui est « ses » sans session — c'est un problème d'identité, pas de droit. b) 403 Forbidden : l'utilisateur est identifié, mais son rôle d'étudiant ne donne pas accès à l'administration ; se reconnecter ne changerait rien, contrairement au 401. c) 400 Bad Request : la demande est mal formée, un champ obligatoire manque ; le client peut corriger et réessayer telle quelle. d) 404 Not Found : la syntaxe est correcte, la ressource demandée n'existe simplement pas. e) 500 Internal Server Error (ou 503 si la panne est connue et temporaire) : la demande était valide, la faute est côté serveur ; le client reçoit un message générique et le détail part dans les journaux. La grille à retenir : 400 = ta demande est fausse, 401 = je ne sais pas qui tu es, 403 = je sais qui tu es et c'est non, 404 = ça n'existe pas, 5xx = c'est ma faute.

QCM d'évaluation

Choisis la bonne réponse pour chaque question. Les corrigés détaillés se trouvent juste après le QCM : ne les regarde qu'après avoir répondu à tout.

1. Dans l'architecture client-serveur du web, qui détient la logique métier et l'accès direct à la base de données ?

A) Le navigateur, car il affiche l'interface

B) Le serveur uniquement

C) La base de données elle-même

2. Quel code de statut signale la création réussie d'une ressource après un POST ?

A) 201 Created

B) 200 OK

C) 204 No Content

3. Que se passe-t-il si la réponse à un fetch vaut 404 ?

A) La promesse est rejetée automatiquement

B) fetch lève une exception réseau

C) La promesse est résolue normalement : il faut tester reponse.ok

4. Pourquoi les mots de passe doivent-ils être stockés avec password_hash() plutôt qu'en SHA1 ?

A) SHA1 produit des empreintes trop longues

B) SHA1 est rapide : un attaquant peut tester des milliards de combinaisons par seconde

C) password_hash chiffre le mot de passe de façon réversible

5. Un étudiant modifie un champ caché « role » de son formulaire pour se déclarer bibliothécaire. Que doit-il se passer ?

A) Il obtient les droits, le formulaire fait foi

B) Rien : le rôle est lu dans la session côté serveur, pas dans les données du client

C) Le navigateur bloque la modification

6. Quelle est la protection définitive contre l'injection SQL ?

A) Filtrer le mot « SELECT » dans les saisies

B) Échapper les apostrophes à la main

C) Les requêtes préparées, qui séparent le code SQL des données

7. Comment un jeton CSRF protège-t-il une action POST ?

A) Il chiffre le contenu du formulaire

B) Le site attaquant ne connaît pas ce secret lié à la session : sa requête forgée est refusée faute de jeton valide

C) Il bloque les requêtes venues d'adresses IP étrangères

8. Pour afficher dans la page un titre de livre venu de l'API, le choix sûr est…

A) textContent, qui insère le texte sans l'interpréter comme du HTML

B) innerHTML, plus rapide

C) document.write, plus simple

9. En déploiement, quelle affirmation est correcte ?

A) On recopie les fichiers modifiés un à un par FTP

B) On active display_errors pour surveiller la production

C) Le code est identique partout ; seuls les paramètres du .env changent selon l'environnement

10. Un utilisateur connecté reçoit 403 sur le point d'entrée de retour des livres. Cela signifie que…

A) Sa session a expiré, il doit se reconnecter

B) Il est identifié mais son rôle ne permet pas cette action

C) Le livre demandé n'existe pas

Corrigés du QCM

1. Réponse B — Le serveur applique les règles de gestion et est le seul à interroger la base ; le navigateur ne fait qu'afficher et demander.

2. Réponse A — 201 Created est le statut précis d'une création réussie ; 200 convient aux lectures et mises à jour, 204 à une réussite sans contenu renvoyé.

3. Réponse C — fetch ne rejette que sur échec réseau. Un 404 ou un 500 arrive comme une réponse ordinaire, à détecter via reponse.ok ou reponse.status.

4. Réponse B — bcrypt/Argon2id (password_hash) sont lents et salés par conception, faits pour les mots de passe ; SHA1, conçu pour la vitesse, se casse en masse. Et un hachage est à sens unique : il ne « déchiffre » rien.

5. Réponse B — Toute donnée venant du client est falsifiable. Le rôle autoritaire est celui écrit par le serveur dans la session au moment de la connexion.

6. Réponse C — Avec une requête préparée, la valeur voyage séparément et n'est jamais interprétée comme du SQL. Les filtres artisanaux finissent toujours par laisser passer un cas.

7. Réponse B — Le jeton est un secret que seul le vrai formulaire possède. La requête forgée, envoyée par le navigateur mais fabriquée ailleurs, ne peut pas le présenter.

8. Réponse A — textContent neutralise toute balise contenue dans la donnée : c'est la défense de base contre le XSS côté client.

9. Réponse C — Code unique + configuration par environnement (.env) = déploiements répétables et retours arrière propres. En production, les erreurs s'affichent… dans les journaux, pas à l'écran.

10. Réponse B — 401 = identité absente ou invalide ; 403 = identité connue mais droit insuffisant — ici, seuls les bibliothécaires enregistrent les retours.

Résumé — les notions importantes

• Full-stack = un client (navigateur), un serveur (logique métier) et une base, qui dialoguent en HTTP : méthodes et URL pour les demandes, codes de statut pour les verdicts.

• Projet structuré : seul public/ est exposé, le métier vit dans src/, les secrets dans un .env jamais versionné — les conventions rendent le projet prévisible et déployable.

• Côté client, fetch + async/await consomment l'API en JSON ; on teste reponse.ok, on gère les erreurs en try/catch, on affiche avec textContent.

• Côté serveur, chaque point d'entrée authentifie, valide, applique le métier, puis interroge MySQL via PDO en requêtes préparées et répond en JSON.

• Authentification : mots de passe hachés (password_hash), session côté serveur régénérée à la connexion, cookie HttpOnly ; les rôles se contrôlent côté serveur, point d'entrée par point d'entrée.

• Sécurité de base : échapper les sorties contre le XSS, requêtes préparées contre l'injection SQL, jeton CSRF vérifié par hash_equals contre les requêtes forgées — jamais de confiance dans le navigateur.

• Déploiement : même code partout, paramètres par .env, rituel répétable avec sauvegarde et retour arrière, HTTPS forcé et erreurs journalisées en production.

Vidéos associées aux leçons

Ces vidéos, en français, complètent les leçons. Liens vérifiés au moment de la publication de ce document.

Leçon 1 — Le protocole HTTP pour les débutants - Théorie + démo

Chaîne : IT-Connect - Florian · Langue : Français · Niveau : Débutant

Une présentation claire du protocole HTTP : requêtes, réponses, versions et ports, suivie d'une démonstration avec un vrai serveur Apache, cURL et Wireshark. Le complément visuel idéal de la leçon 1 pour voir les échanges en texte brut.

https://www.youtube.com/watch?v=gKxKWwikP_c

Leçon 3 — Découvrir le JavaScript_chp-19! L'usage de l'API Fetch

Chaîne : Ronasdev · Langue : Français · Niveau : Débutant à intermédiaire

Un chapitre entier consacré à l'API Fetch : effectuer des requêtes HTTP depuis JavaScript, traiter les réponses et comprendre la différence avec l'ancien XMLHttpRequest. À regarder en codant en parallèle l'exemple de la leçon 3.

https://www.youtube.com/watch?v=MvimkeZcwaE

Leçon 6 — XSS et mots de passe : Le danger du remplissage automatique

Chaîne : Secureaks · Langue : Français · Niveau : Intermédiaire

Un pentester montre concrètement comment une faille XSS peut être exploitée pour voler des identifiants enregistrés dans le navigateur. La démonstration rend tangible le danger que l'échappement des sorties, vu en leçon 6, empêche.

https://www.youtube.com/watch?v=Bt9zuKGLlCc

Leçon 6 — [CS06] Pentesting 1 , démonstration d'attaque Web et sécurisation de l'application Speaky.fr

Chaîne : Julien Code · Langue : Français · Niveau : Intermédiaire

Un test d'intrusion filmé, réalisé avec autorisation sur une vraie petite application web : les failles sont exploitées puis corrigées une à une. Parfait pour voir comment les failles de la leçon 6 s'enchaînent dans la réalité.

https://www.youtube.com/watch?v=y_xTq0AJ2FY

Leçon 7 — Héberger un site web gratuitement avec Netlify

Chaîne : Futura Code · Langue : Français · Niveau : Débutant

La mise en ligne pas à pas d'un site sur une plateforme d'hébergement moderne : préparation des fichiers, déploiement, obtention de l'URL publique. Un premier aperçu concret du geste de déploiement étudié en leçon 7.

https://www.youtube.com/watch?v=ZcMp6nJzVA8

Sources et références

• Programme officiel Licence Informatique, Université IAM (Institut Africain de Management), Bamako — vérifier les intitulés d'UE auprès du secrétariat pédagogique.

• MDN Web Docs (developer.mozilla.org/fr) — les références de HTTP (méthodes, en-têtes, codes de statut) et de l'API Fetch, en français.

• OWASP Cheat Sheet Series (cheatsheetseries.owasp.org) — les fiches de prévention : XSS, CSRF, injection SQL, gestion des sessions et des mots de passe.

• Documentation officielle PHP (php.net) — PDO et requêtes préparées, password_hash/password_verify, sessions et htmlspecialchars.

• Les vidéos listées dans la section « Vidéos associées aux leçons » de ce document (liens vérifiés à la publication).

Fin du cours — Bronbah Tech, Bibliothèque Universitaire IAM. Bon courage pour tes révisions !